```
For every warehouse, calculate:

WarehouseID --> warehouse
Warehouse name --> warehouse
Number of inventory records --> inventory
Total stock --> sum()
Average stock --> avg()
Highest stock in a single inventory record --> max()

Display only warehouses where the average stock per inventory
record is greater than 100. --> having

Sort by total stock descending. --> order by



select
  w.WarehouseID,
  w.Warehouse,
  count(i.WarehouseID) as count_records,
  sum(i.Stock) as total_stock,
  round(avg(i.Stock),2) as avg_stock,
  max(i.Stock) as max_stock
from `e1.warehouses` as w
inner join `e1.inventory` as i
on w.WarehouseID = i.WarehouseID
group by
  w.WarehouseID,
```

```
  w.Warehouse
having avg_stock > 100
order by total_stock desc
```

Find products that:

Exist in the product catalogue
Have inventory records
Have never appeared in any order item --> order_items

Display:

ProductID --> product
ProductName --> product
SupplierID --> product
Number of inventory records --> inventory
Total stock --> inventory

Sort by total stock from highest to lowest.

```
select
  p.ProductID,
  p.ProductName,
  p.SupplierID,
```

```
  count(i.WarehouseID) as no_of_records,
  sum(i.Stock) as total_stock
from `e1.products` as p
inner join `e1.inventory` as i
on p.ProductID = i.ProductID
left join `e1.order_items` as oi
on p.ProductID = oi.ProductID
where oi.ProductID is null
group by
  p.ProductID,
  p.ProductName,
  p.SupplierID
order by total_stock
```

```
For each supplier, calculate:

SupplierName --> supplier
Number of products --> products
Average MRP --> products
Highest MRP --> products
Lowest MRP --> products

Classify each supplier as:

Premium Supplier if average MRP >= ₹30,000
Mid-Range Supplier if average MRP >= ₹15,000
```

```
Budget Supplier otherwise

Display the suppliers with the highest average MRP first.



select
  s.SupplierName,
  count(p.ProductID) as no_of_products,
  avg(p.MRP) as avg_mrp,
  max(p.MRP) as max_mrp,
  min(p.MRP) as min_mrp,
  case
    when avg(p.MRP) >= 30000 then "Premium Supplier"
    when avg(p.MRP) >= 15000 then "Mid-Range Supplier"
    else "Budget Supplier"
  end as supplierCategory
from `e1.suppliers` as s
inner join `e1.products` as p
on s.SupplierID = p.SupplierID
group by  s.SupplierName
order by avg_mrp desc



Find customers who have purchased products from at least 3
different categories.
--> count( distinct categoryid)
Display:

CustomerID --> customers
```

```
Customer name --> customers
Number of different categories purchased --> product
Total quantity purchased --> order_items
Total amount spent --> order_items

Sort by number of different categories purchased from highest to
lowest.



select
  c.CustomerID,
  c.Name,
  count(distinct p.CategoryID) as no_diff_categories,
  sum(oi.Quantity) as total_qty,
  sum(oi.Total) as total_amt
from `e1.customers` as c
inner join `e1.orders` as o
on c.CustomerID = o.CustomerID
inner join `e1.order_items` as oi
on o.OrderID = oi.OrderID
inner join `e1.products` as p
on oi.ProductID = p.ProductID
group by
  c.CustomerID,
  c.Name
having no_diff_categories >= 3
order by no_diff_categories desc
```

```
Find customers who have at least one order associated with a
failed payment.

Display:

CustomerID --> customer
CustomerName --> customer
Number of orders having failed payments --> orders
Total value of those orders --> order_items

Display customers with the highest failed-payment order value
first.



select
distinct status
from `e1.payments`



select
  c.CustomerID,
  c.Name,
  count(distinct o.OrderID) as total_failed_orders,
  sum(oi.Total) as total_value
from `e1.customers` as c
inner join `e1.orders` as o
on c.CustomerID = o.CustomerID
inner join `e1.payments` as p
```

```
on o.OrderID = p.OrderID
inner join `e1.order_items` as oi
on o.OrderID = oi.OrderID
where p.Status = "Failed"
group by
  c.CustomerID,
  c.Name
order by total_value desc



Find employees who have handled at least one order whose total
value is greater than ₹75,000.

Display:

EmployeeID --> employee
EmployeeName --> employee
Number of high-value orders --> orders
Total value of high-value orders --> order_items
Average high-value order amount --> order_items
having

Sort by total high-value sales from highest to lowest.



select
  e.Name,
  e.EmployeeID,
```

```
  count(distinct o.OrderID) as no_of_orders,
  sum(oi.Total) as high_value_sales_amt,
  avg(oi.Total) as avg_high_value_sales_amt
from `e1.employees` as e
inner join `e1.orders` as o
on e.EmployeeID = o.EmployeeID
inner join `e1.order_items` as oi
on o.OrderID = oi.OrderID
group by
  e.Name,
  e.EmployeeID
having high_value_sales_amt > 75000
order by high_value_sales_amt desc
```

Find products whose MRP is greater than the average MRP of all products.

Display:

ProductID
ProductName
CategoryName
MRP

Sort from highest MRP to lowest.

For each warehouse, calculate total stock and classify the warehouse as:

High Stock → total stock >= 5,000
Medium Stock → total stock >= 2,500
Low Stock → total stock < 2,500

Display:

WarehouseID
Warehouse name
Total stock
Average stock
Stock classification

Sort warehouses by total stock descending.

Find customers who have placed orders on more than 2 different order dates.

Display:

CustomerID
CustomerName

```
Number of different order dates
Total number of orders
Total quantity purchased

Sort by number of different order dates from highest to lowest.
```